

UNIT 4: MEMORY MANAGEMENT & VIRTUAL MEMORY

Course Pattern: TE IT - 2024 Pattern (SPPU)
Status: Publication-Grade Theory Guide

Academic Term: Semester V (2026)
Course Code: PCC-301-ITT

Syllabus Mapping & Unit Structure

The following table outlines the curriculum mapping and priority weightage for Unit 4.

Syllabus Topic	In-Depth Exam Coverage	Priority / Weightage
Memory Partitioning	Fixed vs Dynamic Partitioning, Internal & External Fragmentation, Compaction, Buddy System (power-of-2 allocation math).	High Priority (6-8 Marks Qs)
Contiguous Allocation & Placement	Memory Management Unit (MMU), Logical vs Physical Address Space, Placement Strategies (First Fit, Best Fit, Worst Fit, Next Fit).	High Priority (8-10 Marks Qs)
Paging & Segmentation	Page & Frame mapping, Page Table Entries (PTE), 2-Level & Inverted Paging, Segmentation, Hardware Address Translation math.	High Priority (10-12 Marks Qs)
Virtual Memory & TLB	Demand Paging, Page Fault Lifecycle, Translation Lookaside Buffer (TLB), Hit Ratio & Effective Access Time (EAT) calculations.	High Priority (8-10 Marks Qs)
Page Replacement & Thrashing	Page Replacement Algorithms (FIFO, Optimal, LRU, MRU), Belady's Anomaly, Thrashing, Working Set Model, Page Fault Frequency.	High Priority (10-12 Marks Qs)

Memory Management Requirements and Partitioning

Five Core Requirements of Memory Management

- Relocation:** The OS must dynamically map logical process instructions to physical RAM addresses at runtime as processes are loaded, swapped out, and reloaded.
- Protection:** User processes must be hardware-restricted from accessing or modifying the physical memory space of other processes or the OS kernel without permission.
- Sharing:** The OS must permit multiple concurrent processes to safely share a single copy of common memory regions (e.g., shared code libraries, shared memory IPC segments).
- Logical Organization:** Memory is organized as modular, linear user segments (main program, modules, data arrays, stacks) corresponding to human program structure.
- Physical Organization:** The OS manages the physical hierarchy separating fast, volatile primary RAM from slow, non-volatile secondary storage (disk/SSD).

Memory Partitioning Schemes: Fixed vs. Dynamic Partitioning

1. Fixed (Static) Partitioning:

- Main memory is divided into fixed-size static regions (equal or unequal sizes) at system initialization.
- **Internal Fragmentation:** Occurs when a process allocated to a partition is smaller than the partition size; the wasted internal memory cannot be used by any other process.

2. Dynamic (Variable) Partitioning:

- Memory partitions are created dynamically, allocating the exact amount of RAM required by a process.
- **External Fragmentation:** As processes terminate, memory becomes broken into small, non-contiguous free gaps. Although total free RAM is sufficient, no single contiguous hole can satisfy an incoming request.
- **Compaction:** OS shuffle mechanism that relocates processes in RAM to merge all free spaces into one large contiguous block (incurs high CPU relocation overhead).

3. The Buddy System:

- Memory blocks are allocated in powers of two (2^K). If a request of size s requires a block, the OS splits a free block of size 2^U into two equal “buddies” of size 2^{U-1} until the smallest power-of-two block $2^K \geq s$ is reached.
- When a process terminates, adjacent free buddies of identical size are automatically merged back into 2^{K+1} .

Parameter	Fixed Partitioning	Dynamic Partitioning	Buddy System
Partition Size	Static, predetermined size.	Variable, exact process size.	Powers of two (2^K bytes).
Fragmentation Type	Severe Internal Fragmentation.	Severe External Fragmentation.	Moderate Internal & External.
Management Overhead	Very low.	High (Requires Compaction).	Moderate (Fast power-of-2 merging).
Degree of Multiprogramming	Limited by fixed partition count.	Dynamic, limited only by total RAM.	Flexible and highly responsive.

Source: Stallings, Ch 7, p. 305-330 | Source: Silberschatz, Ch 8, p. 345-365

Contiguous Memory Allocation and Placement Strategies

Memory Management Unit (MMU) and Address Translation

The CPU generates **Logical (Virtual) Addresses**, whereas physical RAM memory controller hardware understands **Physical Addresses**.

- **Relocation Register (Base Register):** Holds the starting physical RAM address of the active process.
- **Limit Register:** Specifies the exact size bound of the process memory space.
- **Hardware Translation Formula:** Physical Address = Logical Address + Relocation Register. If Logical Address $\geq$ Limit Register, the MMU generates a hardware **Segmentation Fault / Trap**.

Dynamic Memory Placement Strategies

1. **First Fit:** Allocates the **first** available free hole from the beginning of RAM that is large enough. Fast, low search overhead.
2. **Best Fit:** Searches the entire memory list to allocate the **smallest** hole that is large enough. Leaves behind tiny unusable residual holes (high external fragmentation).
3. **Worst Fit:** Allocates the **largest** available free hole. Leaves behind larger remaining holes, but requires searching the entire memory list.
4. **Next Fit:** Similar to First Fit, but starts searching from the location of the last allocated block rather than the beginning of memory.

Source: Silberschatz, Ch 8, p. 350-370 | Source: Deitel, Ch 5, p. 180-205

Non-Contiguous Allocation: Paging and Segmentation

Fundamental Principles of Paging

Paging is a non-contiguous memory management scheme that completely eliminates External Fragmentation.

1. Pages and Frames:

- **Physical RAM** is divided into fixed-size physical blocks called **Frames**.
- **Logical Memory** of a process is divided into equal-sized blocks called **Pages**.
- Frame size is always equal to Page size (typically 4 KB = 4096 bytes).

2. Page Table Architecture: The OS maintains a dedicated **Page Table** for each process. The Page Table maps logical Page Numbers (p) to physical Frame Numbers (f).

3. Hardware Address Translation Math: For a m -bit logical address space and 2^n -byte page size:

- **Page Number (p):** High-order $m - n$ bits ($p = \frac{\text{Logical Address}}{\text{Page Size}}$).
- **Page Offset (d):** Low-order n bits ($d = \text{Logical Address} \bmod \text{Page Size}$).
- **Physical Address Formula:** Physical Address = $(f * \text{Page Size}) + d$.

CPU Logical Address [$pmid$] → Page Table Lookup at index p → Retrieve Frame Number f

Combine Frame Number f with Offset d → Physical Memory Address [$fmid$]

Figure 4.1: Hardware Address Translation with Paging and Page Table

Hierarchical (Two-Level) and Inverted Page Tables

1. **Two-Level Paging:** On 32-bit systems with 4 KB pages, a single page table requires 4 MB of contiguous RAM per process. Two-Level paging splits the page number into an **Outer Page Table** (p_1) and an **Inner Page Number** (p_2), paging the page table itself.
2. **Inverted Page Table:** Rather than maintaining one page table per process, an Inverted Page Table has a single entry for each physical RAM frame in the system, indexed by $l \angle \text{PID}, pr \angle$. Drastically reduces page table memory consumption.

Segmentation and Comparison with Paging

Segmentation divides logical memory into variable-length user-centric segments (Main program, Functions, Data arrays, Stack).

- **Segment Table:** Maps Segment Number (s) to Segment Base Address (b) and Segment Limit (L).
- Address translation checks if Offset $d < L$; if valid, Physical Address = $b + d$.

Parameter	Paging	Segmentation
Block Size	Fixed physical size (e.g., 4 KB).	Variable logical length.
User Visibility	Completely transparent to user/programmer.	Visible to programmer (logical modules).

Parameter	Paging	Segmentation
Fragmentation	Internal fragmentation only; zero external.	External fragmentation; zero internal.
Hardware Table	Page Table (maps page p to frame f).	Segment Table (maps segment s to base b , limit L).

Source: Stallings, Ch 8, p. 340-370 | Source: Tanenbaum, Ch 3, p. 190-220

Virtual Memory and Translation Lookaside Buffer (TLB)

Fundamental Principle of Virtual Memory & Demand Paging

Virtual Memory decouples logical user memory from physical RAM, allowing execution of processes whose total memory space exceeds available physical RAM.

1. **Demand Paging:** Pages are loaded into physical RAM only when referenced during execution (Lazy Swapper / Pager).
2. **Page Table Entry (PTE) Bits:**
 - **Valid / Invalid (Present) Bit:** 1 if page is currently loaded in RAM; 0 if page resides on secondary disk.
 - **Dirty (Modify) Bit:** 1 if page content was modified in RAM (must be written back to disk on replacement).
 - **Protection Bits:** Read, Write, Execute access permissions.

Page Fault Handling Lifecycle

A **Page Fault** occurs when a process attempts to access a page marked invalid (Present bit = 0).

Memory Access Instruction → PTE Valid Bit = 0 → CPU Hardware Trap to OS Kernel

Find Free RAM Frame → Read Page from Disk into Frame → Update PTE (Set Valid=1, Frame=f)

Restore Saved Process Context → Restart Faulting Instruction

Figure 4.2: Step-by-Step Page Fault Handling Lifecycle Sequence

Translation Lookaside Buffer (TLB) and Effective Access Time (EAT)

Because paging requires two RAM accesses per instruction (one for Page Table, one for Data), hardware designers add a fast hardware associative cache called the **Translation Lookaside Buffer (TLB)**.

Logical Page Number p → Parallel TLB Search

TLB Hit: Retrieve Frame Number f immediately from TLB ($t_{\text{TLB}} + t_{\text{RAM}}$)

TLB Miss: Access Page Table in RAM → Update TLB ($t_{\text{TLB}} + 2 * t_{\text{RAM}}$)

Figure 4.3: Hardware Address Translation with TLB Cache

Effective Access Time (EAT) Formula

Let α be the TLB Hit Ratio, ε be the TLB search time, and m be physical RAM access time: $EAT = \alpha * (\varepsilon + m) + (1 - \alpha) * (\varepsilon + 2m)$

Source: Silberschatz, Ch 9, p. 390-420 | Source: Tanenbaum, Ch 3, p. 210-240

Page Replacement Algorithms and Thrashing

Page Replacement Algorithms Overview

When a page fault occurs and no physical RAM frames are free, the OS must select a **Victim Page** to replace.

Algorithm	Replacement Rule	Key Features	Major Drawbacks & Anomalies
FIFO	Replaces the oldest page loaded into RAM.	Simple queue implementation.	Belady's Anomaly: Page faults increase when allocated frames increase!
Optimal (OPT)	Replaces page that will not be used for longest time in future.	Lowest possible page fault rate.	Impossible to implement in real systems (requires future knowledge).
LRU	Replaces page that has not been used for longest past time.	Excellent practical approximation of OPT.	Requires hardware stack or 64-bit counter per PTE.
MRU	Replaces page most recently used.	Useful for specific file scanning patterns.	High page fault rate for general workloads.

Belady's Anomaly in FIFO

Belady's Anomaly is the counter-intuitive phenomenon where increasing the number of allocated RAM frames results in an **increase** in the total number of page faults for the FIFO algorithm. (LRU and Optimal algorithms are stack algorithms and are immune to Belady's anomaly).

Thrashing and Working Set Model

Thrashing occurs when a process does not have enough physical RAM frames to support its active pages. The process spends more time swapping pages in and out of disk than executing instructions, causing CPU utilization to collapse to near zero.

1. **Working Set Model:** Based on locality of reference. The Working Set $W(t, \Delta)$ is the set of pages referenced by the process during the most recent time window Δ .
2. **Page Fault Frequency (PFF):** Establishes upper and lower bounds on page fault rates. If fault rate exceeds upper threshold, OS allocates more frames to process; if fault rate falls below lower threshold, OS reclaims unused frames.

Source: Stallings, Ch 8, p. 375-405 | Source: Silberschatz, Ch 9, p. 420-450